



Common Patterns

Topological Sort 🏰

Ordering tasks with dependencies using Kahn's algorithm and DFS — a pattern that appears whenever problems involve build systems, scheduling, or prerequisites.

The whole workflow in about ten minutes: how to recognize this pattern, drive the AI through Kahn's, handle the cycle traps, and verify the result. Chapters let you skip to whatever you need. The written deep dive is below.

Run `pip install pandas` and you'll watch it pull down NumPy first, then a dozen smaller things, and only install pandas itself at the end. If pip went in random order, half the imports inside pandas would blow up because their dependencies hadn't loaded yet. So pip figures out an order where nothing gets installed before the things it depends on.

That ordering move has a name: **topological sort**. The structure underneath is a directed acyclic graph (DAG), a graph where edges have a direction (A points to B, not back) and following the arrows never loops you back to where you started. Topological sort lays the nodes out in a line such that every arrow points forward.

It shows up wherever one thing has to happen "after" another: build systems, course schedulers, task pipelines, spreadsheet recalc. If the name is new to you, that part doesn't matter. What matters is recognizing the dependency-ordering shape when a problem describes it, and being able to steer the AI through the implementation once you do.

Recognizing the pattern

Topological sort problems almost never announce themselves. They arrive dressed up as build systems, task schedulers, dependency installers, course planners, or compilation pipelines, and your job is to see the DAG underneath. The interviewer is watching whether you can name the structure before you start coding.

If you see any of these signals, your pattern-matching instinct should immediately flag topological sort:

- **Vocabulary from the problem statement.** "Prerequisites," "dependencies," "ordering," "before/after," "must complete first," "blocks." These are giveaways.

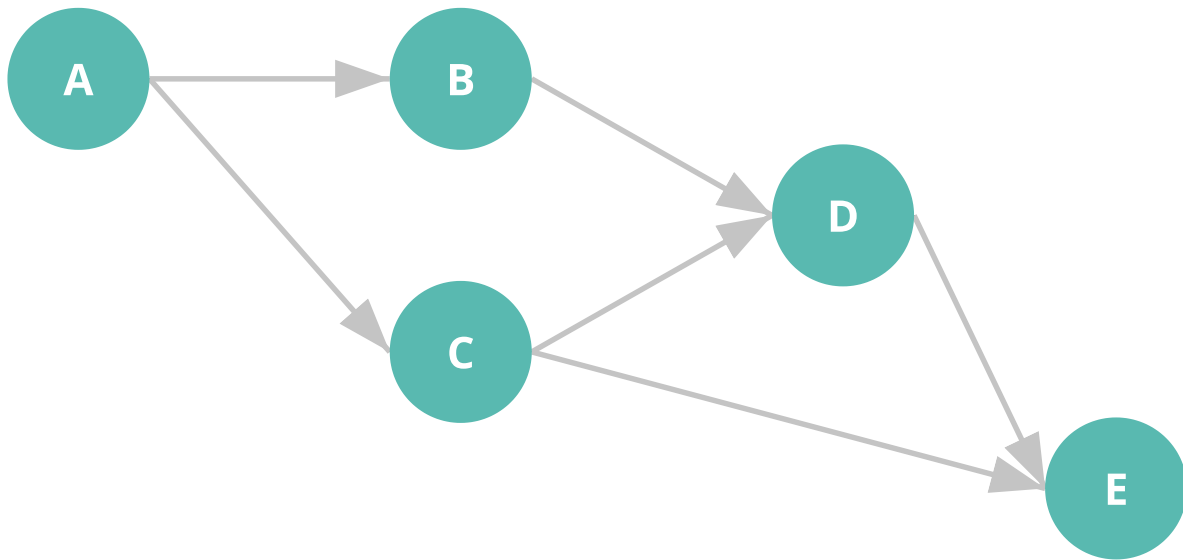
- **A list of pairs that describe a "must come before" relationship.** Course A requires course B. Package X depends on Y. Step 1 must finish before Step 2. Any time the input is a list of these ordered pairs, you're being handed a graph and asked to linearize it.
- **A question about feasibility.** "Is there an order in which all of these can run?" If the answer might be "no because there's a cycle," topological sort is the right tool — it both produces the order and detects the impossibility.
- **A scheduling problem with prerequisites.** Parallel task execution, semester planning, build pipelines. The topological order is the scaffolding; the actual answer often involves a second pass on top of it.

Say the pattern out loud when you spot it: "This looks like a dependency graph — I'd model it as a DAG and use topological sort to find a valid ordering." That one sentence tells the interviewer you've identified the pattern, and it gives them confidence you're heading in the right direction.

What is a topological ordering?

A topological ordering is a linear arrangement of nodes in a directed acyclic graph (DAG) where for every directed edge from node A to node B, A appears before B in the ordering. That's it. You're just lining things up so that nothing comes before its dependencies.

Think about a build system. You've got six modules, and some of them depend on others. Module D needs modules B and C to be built first. Module B needs module A. You can't just build them in any random order — you need an ordering that respects all the dependency arrows.



Arrows point from dependency to dependent. A must come before B and C.

One valid topological ordering here is A, B, C, D, E. Another is A, C, B, D, E. Both are correct. Topological sort doesn't produce a unique answer unless the graph happens to have only one valid path. The key constraint is just that every node appears after all of its dependencies.

This is worth internalizing because it comes up in interviews. If the problem says "return any valid ordering," you don't need to worry about which specific ordering your algorithm produces. Read the requirements carefully.

The "acyclic" part is crucial. If module A depends on B and B depends on A, there's no valid ordering. Cycles make topological sort impossible. We'll come back to that.

Algorithms

Two algorithms get you a topological ordering: Kahn's (BFS-style) and a DFS-based approach. Kahn's is the one to reach for in an interview. The DFS version is here mostly so you can recognize it if the AI produces it, and so you understand the tradeoffs.

Kahn's algorithm (BFS approach)

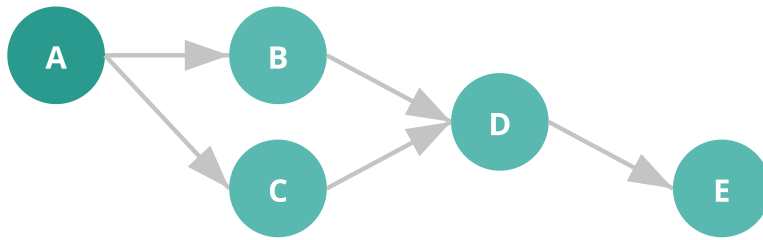
Kahn's algorithm is the most intuitive way to think about topological sort. The idea is to find everything that has no dependencies, process it, then remove it from the graph and repeat. It's exactly what you'd do if you were manually figuring out a build order on a whiteboard.

1. Calculate the in-degree (number of incoming edges) for every node
2. Add all nodes with in-degree 0 to a queue — these have no dependencies and are safe to process first
3. Pull a node from the queue, add it to the result
4. For each neighbor of that node, decrement its in-degree. If the in-degree hits 0, add it to the queue
5. Repeat until the queue is empty

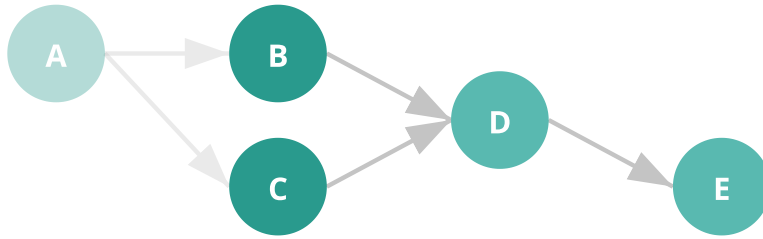
If you processed all nodes, you have a valid topological ordering. If some nodes are left over, there's a cycle (coming up!).

The "in-degree" terminology sounds academic, but the concept is very easy: it's just the count of how many things a node depends on. When that count reaches zero, all dependencies are satisfied and the node is ready to go.

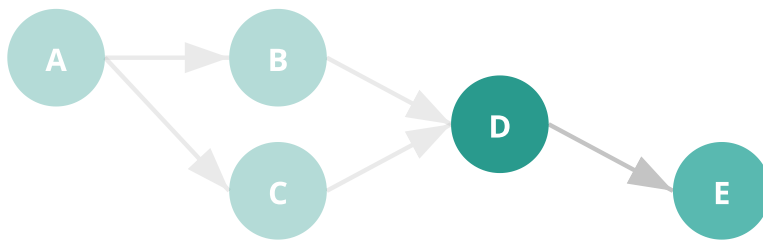
Step 1: A has in-degree 0 — process it



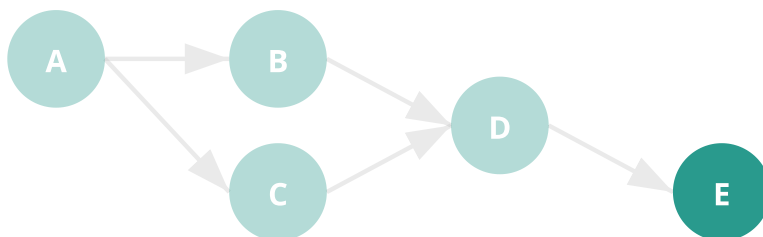
Step 2: B and C now have in-degree 0 — process both



Step 3: D now has in-degree 0 — process it



Step 4: E now has in-degree 0 — process it. Done!



Processed

Waiting

Low opacity = done

Result: A, B, C, D, E

 Kahn's Algorithm (BFS Topological Sort)

Python  

```
from collections import deque

def topological_sort_kahn(num_nodes, edges):
    adj = [[] for _ in range(num_nodes)]
    in_degree = [0] * num_nodes
```

```

for src, dst in edges:
    adj[src].append(dst)
    in_degree[dst] += 1

queue = deque()
for node in range(num_nodes):
    if in_degree[node] == 0:
        queue.append(node)

result = []
while queue:
    node = queue.popleft()
    result.append(node)
    for neighbor in adj[node]:
        in_degree[neighbor] -= 1
        if in_degree[neighbor] == 0:
            queue.append(neighbor)

if len(result) != num_nodes:

```

The time complexity is $O(V + E)$ where V is vertices and E is edges. You visit every node once and process every edge once. Space is also $O(V + E)$ for the adjacency list.

Notice how naturally Kahn's algorithm detects cycles. If the graph has a cycle, those nodes will never reach in-degree 0, so they'll never enter the queue. You just check whether your result contains all nodes at the end. If it doesn't, there's a cycle somewhere.



If you see words like "prerequisites", "dependencies", "ordering", or "before/after" in a problem description, your pattern-matching instinct should immediately flag topological sort.

DFS-based topological sort

There's another approach (spoiler: we don't recommend it) that uses depth-first search. Instead of peeling off nodes with no dependencies from the front, you recurse as deep as possible and add nodes to the result on your way back up. The result comes out in reverse order, so you reverse it at the end (or prepend to a list, or use a stack).

Why does this work? In a DFS, a node doesn't finish until all of its descendants finish. So if you write down nodes in the order they finish, anything a node depends on shows up earlier in the recursion and gets written down first. Reverse the list at the end and you've got a valid ordering.



DFS-Based Topological Sort

Python



```
def topological_sort_dfs(num_nodes, edges):
    adj = [[] for _ in range(num_nodes)]
    for src, dst in edges:
        adj[src].append(dst)

    UNVISITED, IN_PROGRESS, DONE = 0, 1, 2
    state = [UNVISITED] * num_nodes
    result = []

    def dfs(node):
        if state[node] == IN_PROGRESS:
            return False
        if state[node] == DONE:
            return True

        state[node] = IN_PROGRESS
        for neighbor in adj[node]:
            if not dfs(neighbor):
                return False
        state[node] = DONE
        result.append(node)
        return True

    for node in range(num_nodes):
        if state[node] == UNVISITED:
            if not dfs(node):
                return None

    result.reverse()
    return result
```

The three-state tracking (UNVISITED, IN_PROGRESS, DONE) is doing double duty here. It prevents revisiting nodes we've already fully processed, and it detects cycles: if we encounter a node that's IN_PROGRESS, we've found a back edge, which means there's a cycle.

Walk through a small example to see why this works. Starting from node A, we mark it `IN_PROGRESS` and recurse into its neighbors. When we finish all of A's descendants, we mark A as `DONE` and append it to the result. Because A finishes after everything reachable from A, it ends up later in the result list. Reversing at the end puts it first, exactly where a node with no dependencies belongs.

Kahn's vs. DFS: when to reach for which

Both run in $O(V + E)$ time. Both produce valid topological orderings. For interviews, **just use Kahn's**. It's easier to understand, easier to explain, and gives you cycle detection for free. The BFS structure means you process nodes level by level, which maps naturally to "rounds" of work — handy if the problem also asks about parallelism (which tasks can run simultaneously?).

The reason this matters in an AI-enabled interview is simple. The AI can write the algorithm; the bar is being able to read what it produced, explain it, and catch bugs. "Process things with no dependencies, remove them, repeat" is something you can keep in your head while you skim Kahn's output. DFS-based topological sort asks you to track post-order finishing times *and* three-state cycle detection at the same time, and most interview problems already have other stuff layered on top.



DFS-based topological sort might make sense if you're already doing a graph traversal for another reason and can piggyback the ordering on top of it. But if the problem is purely "find a valid ordering," Kahn's is the better choice to direct your AI toward. You'll understand the output faster and catch bugs more easily. Just use Kahn's!

Cycle detection

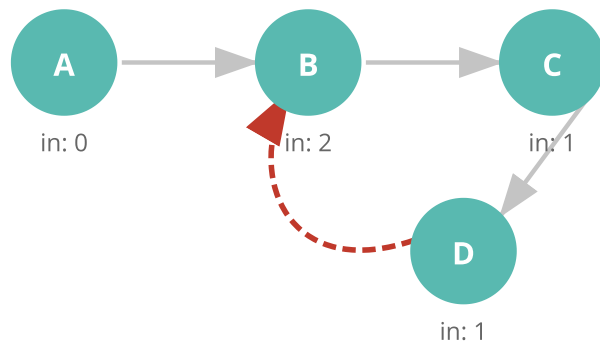
If the graph isn't a DAG — if it has cycles — topological sort is impossible. No valid ordering exists. Think about it: if A depends on B and B depends on A, which one goes first? Neither can.

Detecting cycles is part of most dependency-resolution problems, either explicitly ("report an error if there are circular dependencies") or implicitly (the algorithm just needs to handle it gracefully). In real-world systems, circular dependencies are bugs — package managers refuse

to install, build systems throw errors, and compilers complain. In interview problems, cycle detection is often the twist that separates a complete solution from a partial one.

With Kahn's algorithm, cycle detection is built in. If `len(result) < num_nodes` after the algorithm finishes, some nodes were never added to the queue because their in-degree never reached 0. Those nodes are part of one or more cycles.

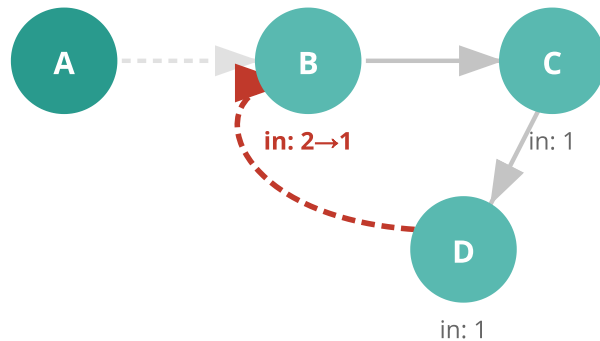
Graph with a cycle: D depends on B, but B depends on D (through C)



Cycle: B → C → D → B

Dashed red arrow creates circular dependency

Step 1: A has in-degree 0 — process it. Decrement B's in-degree.



B never reaches in-degree 0 because D still points to it, but D is waiting on C, which is waiting on B.

Deadlock.

Queue is empty. Result: [A] — only 1 of 4 nodes. Cycle detected!

With DFS, you detect cycles by finding back edges — encountering a node that's currently in the IN_PROGRESS state during traversal.



When you prompt the AI for topological sort, tell it to return the nodes involved in the cycle rather than just returning `None` or throwing a generic error. Cycle detection comes up constantly in dependency-resolution problems, and when your solution hits a cycle in a

large test case, "cycle detected" tells you nothing useful. "Cycle detected: $B \rightarrow C \rightarrow D \rightarrow B$ " tells you exactly where the problem is.

Common variations you'll encounter

Pure "give me a topological ordering" is rare in real interviews. The pattern shows up in disguise, and often the topological sort is just one piece of a larger problem.

Parallel task scheduling. Given tasks with dependencies and durations, find the minimum total time to complete everything assuming unlimited parallelism. This is the critical path problem. You run topological sort, then do a forward pass computing the earliest start time for each task (the maximum completion time among all its dependencies). The answer is the maximum completion time across all tasks. Kahn's is especially natural here because its level-by-level processing maps directly to "rounds" of parallel execution.

Course scheduling with semesters. A variant of parallel scheduling: given course prerequisites and a limit on courses per semester, find the minimum number of semesters. Same idea as above, but you cap the number of nodes you process per "level" in Kahn's algorithm. See the [Course Schedule](#) problem for the canonical version.

Detecting if a unique ordering exists. If at any point during Kahn's algorithm the queue contains more than one node, multiple valid orderings exist. If the queue always has exactly one node, the ordering is unique. This is a common follow-up question — the [topological sort problem walkthrough](#) covers it in depth.

Longest path in a DAG. Process nodes in topological order, and for each node, update the distances of its neighbors. Unlike shortest path algorithms that work on general graphs, longest path in a DAG is solvable in $O(V + E)$ precisely because topological ordering lets you process each node exactly once, after all paths leading to it are finalized.

The unifying theme across all these variations: model the problem as a directed graph, sort it topologically, then do whatever computation you need in that order. The sort is rarely the whole answer — it's the scaffolding that makes the rest of the answer efficient.

Prompting the AI

Before you prompt, narrate the plan to your interviewer in topological-sort vocabulary: "adjacency list, in-degrees, Kahn's, handle cycles." That sentence tells them you know the shape of the algorithm before the AI writes a line of it.

When you actually prompt the AI, leave a little room. "Find a valid ordering of these tasks given the dependencies, and tell me if there's a cycle" gives the model enough direction to do the right thing while leaving open the possibility that it sees something you didn't. Over-constraining to a specific algorithm sometimes traps you if you guessed wrong about the problem shape.

Two pattern-specific moves worth knowing:

- **Ask for cycle nodes, not a boolean.** When the AI hits a cycle, default behavior is to throw or return `None`. That tells you nothing useful when a large test case fails. "If you detect a cycle, return the nodes involved in it" is a one-line addition to your prompt that turns "cycle detected" into " $B \rightarrow C \rightarrow D \rightarrow B$." The difference is thirty seconds of debugging vs. five minutes staring at the output.
- **Be explicit about input format.** "The input is a list of (`prerequisite`, `dependent`) pairs" or "the input is an adjacency map." Topological sort code is short — the bugs are almost always in how you parsed the input into the graph, not the algorithm itself.



A common mistake is jumping straight to code without establishing the graph structure first. Before you or the AI write a single line, make sure you can answer: what are the nodes, what are the edges, and where does the input data come from? Getting the graph construction wrong will silently produce a wrong ordering, and debugging a wrong topological sort is much harder than debugging a wrong graph.

Verifying the AI's code

Topological sort is short and the algorithm is well-known, so the AI almost always gets the inner loop right. The bugs live in the boundary cases. Check these three:

1. **Cycle handling.** Run a small input with a cycle. Does the code return the partial ordering and silently move on, or does it correctly flag the cycle? If `len(result) < num_nodes` isn't checked, you have a silent-failure bug waiting to ship.

2. **Disconnected nodes.** What about a node with no dependencies and no dependents? It should still appear in the output. The in-degree-zero queue should pick it up at the start; if the AI built the adjacency list only from edges, isolated nodes can get dropped.
3. **The graph construction itself.** Print the adjacency list and in-degree map for a small input before running the algorithm. If those two are wrong, the rest of the code can be perfect and you'll still get the wrong answer.

Test with a four-node cyclic example first ($A \rightarrow B \rightarrow C \rightarrow A$, plus an isolated D). If the code returns a useful cycle report and includes D in any non-cyclic prefix, the boundaries are sound.

When to use vs. alternatives

Topological sort is the right tool when the problem is fundamentally about ordering by dependency. A few neighbors that get confused with it:

- **Generic graph traversal.** If the problem doesn't care about ordering and just asks "can I reach X?" or "what are the connected components?", that's graph search, not topological sort.
- **Shortest path on a DAG.** This is built *on top of* topological sort, not instead of it. Sort the DAG topologically, then relax edges in order — that's the algorithm.
- **General scheduling without dependencies.** If tasks don't depend on each other but you need to maximize throughput or minimize cost, you're in greedy territory.
- **Tasks with dependencies *and* an optimization goal.** Topological sort gives you the order; the optimization step on top usually wants DP. The classic example is critical path analysis — topo sort the DAG, then DP a forward pass for earliest start times.

If the problem has both ordering and optimization, lead with topological sort and layer the optimization on top in the sorted order. That's almost always cheaper than reaching for a general-purpose algorithm.

What interviewers expect

A few signals on these problems:

- **You name the pattern in the right vocabulary.** "Dependency graph," "DAG," "topological sort" — said before you start coding, in response to the problem statement, not in response to "what algorithm are you using?"
- **You think through representation, edge cases, and algorithm choice before writing code.** Adjacency list, in-degrees, Kahn's, cycle handling. That sentence to the interviewer is the move.
- **You handle cycles explicitly.** Either the problem guarantees no cycles (read carefully and say so) or you need to detect and report them. Skipping cycle handling is a common silent failure that interviewers specifically probe for.
- **You see the second half of the problem.** Most dependency problems have a step that comes after the sort: longest path, critical path, parallel scheduling, value propagation. Topological sort is the foundation; the real answer requires a second step. Naming that second step ("now I'd do a forward pass to compute earliest start times") is what separates "I know the algorithm" from "I see the problem."



Interviewers don't expect you to have topological sort memorized. They expect you to recognize when a problem is fundamentally about dependency ordering, choose an appropriate approach, and implement it correctly. Saying "I haven't implemented this recently, but the idea is to repeatedly process nodes with no remaining dependencies" is a perfectly strong signal.

Putting it together

Topological sort problems collapse into the same arc every time. You recognize the dependency-ordering shape, name it out loud, sketch the graph representation, prompt the AI for Kahn's with explicit cycle handling, and verify the boundaries (cycles, disconnected nodes, graph construction).

The implementation is mechanical once the pattern is named. The hard part — and the part interviewers are grading — is the recognition. Pure "give me a topological ordering" is rare; the pattern shows up as one piece of a larger problem, and the candidates who do well are the ones who see it under the build system, the course scheduler, or the task pipeline.

Test Your Knowledge

Take a quick 15 question quiz to test what you've learned and identify gaps.

 Start Quiz